TracePay: A Blockchain-Based Supply-Chain Trade Finance Platform with Oracle-Priced Escrow, Milestone Verification, and On-Chain Invoice Financing

Karen Elisha Chezhiyan
School of Computer Science and Engineering
RV University
Bangalore, India
karenelishachezhiyanbtech24@rvu.edu.in

Name
School of Computer Science and Engineering
RV University
Bangalore, India
mail

Name
School of Computer Science and Engineering
RV University
Bangalore, India
mail

***Abstract*—Global supply-chain trade finance remains plagued by information asymmetry, counterparty risk, slow settlement cycles, and the high cost of invoice-factoring intermediaries. This paper presents *TracePay*, a decentralized application (DApp) that encodes a complete trade-finance lifecycle inside a single Ethereum smart contract. TracePay combines five tightly integrated mechanisms: (1) keccak256 invoice hashing for duplicate-prevention and off-chain privacy; (2) Chainlink ETH/USD oracle-priced escrow with freshness validation; (3) role-gated logistic milestone recording (Dispatched → Delivered) with IPFS evidence CIDs; (4) Delegation-of-Authority (DoA) multi-signature approval for high-value orders exceeding a configurable USD threshold; and (5) on-chain invoice factoring that grants suppliers immediate working capital at a configurable advance ratio (default 90%), with interest settled atomically through a pull-payment waterfall. The contract is implemented in Solidity 0.8.24 using OpenZeppelin's `AccessControl` and `ReentrancyGuard` primitives, and is validated by 16 Hardhat test cases (6 functional, 10 negative/revert) all passing with 100% function coverage. Gas benchmarks show deployment at 7.4% of the Ethereum block limit, and per-transaction costs ranging from 30 628 to 181 063 gas units.**

***Index Terms*—blockchain, smart contract, trade finance, supply chain, Ethereum, Chainlink oracle, escrow, invoice factoring, decentralised finance (DeFi), role-based access control (RBAC), delegation of authority, Solidity**

I. Introduction

Trade finance is the financial backbone of international commerce, bridging the gap between a buyer's need to defer payment and a supplier's need for immediate liquidity [1]. The traditional model relies on letters of credit, bank guarantees, and invoice-factoring intermediaries, each introducing delays of 5–30 days and fees of 2–8% of the transaction value [2]. Worse, 45% of small and medium enterprises (SMEs) report rejected or delayed trade-finance applications due to insufficient credit history or documentation errors [3].

Blockchain technology—and Ethereum smart contracts in particular—offer a path toward trust-minimised, programmable settlement in which payment is released automatically when verifiable conditions are met [4], [5]. Several platforms have explored this space [6]–[8], yet most operate in permissioned consortium settings and expose limited programmability.

We present **TracePay**, an open, permissionless DApp that:

- Encodes the entire buyer–supplier–financier lifecycle in a single auditable smart contract (∼220 SLOC of Solidity).
- Prices escrow in USD via live Chainlink price feeds, insulating both parties from ETH volatility at the moment of commitment.
- Records IoT-verifiable logistics milestones on-chain, linking each to an IPFS content-identifier (CID) for tamper-evident evidence.
- Enforces a configurable Delegation-of-Authority (DoA) policy that requires n-of-m finance approvers before high-value funds are released.
- Enables on-chain invoice factoring so that suppliers can obtain working capital immediately, without waiting for delivery confirmation.

The remainder of this paper is organised as follows. Section II surveys related work. Section III presents the system architecture and protocol design. Section IV details the implementation. Section V reports gas consumption and test results. Section VI analyses security properties. Section VII discusses limitations and future work, and Section VIII concludes.

II. Background and Related Work

A. Blockchain in Trade Finance

The application of distributed ledger technology to trade finance has been widely studied [9]–[11]. Pinna and Ruttenberg

[12] identified finality, atomicity, and programmability as the key properties blockchain brings to financial settlement. Consortium platforms such as we.trade [7] and Contour [8] demonstrated feasibility but suffered from high governance overhead and lack of public accessibility.

B. Decentralised Escrow

Escrow smart contracts have been proposed for freelance work [13] and real-estate transactions [14], yet few anchor the locked amount to a USD-denominated invoice, preferring instead to specify ETH directly. This exposes parties to currency risk: a 20% ETH price move during the settlement window can erode half a month's supplier margin.

C. Oracle Integration

Connecting on-chain contracts to off-chain real-world data (prices, events) is the oracle problem [15], [16]. Chainlink decentralised oracle networks provide aggregated, tamper-resistant price feeds with on-chain freshness metadata [17], [18]. TracePay leverages the `AggregatorV3Interface` to obtain a validated ETH/USD price with strict stale-data detection.

D. Invoice Financing on Blockchain

Tokenised receivables and on-chain factoring have been explored by Centrifuge [19], TradeTrust [20], and fraud-resilient factoring protocols [21]. Our contribution differs in that financing is integrated directly into the escrow lifecycle rather than operating as a secondary token market, eliminating the need for a separate ERC-20 token or bonding curve.

III. System Design

A. Actors and Roles

TracePay defines four actor types, managed via OpenZeppelin's Role-Based Access Control (RBAC):

- **Buyer**: Creates and funds orders; may claim a refund if delivery is missed.
- **Supplier**: Receives payment upon confirmed delivery; may request early financing.
- **Verifier** (`VERIFIER_ROLE`): A trusted logistics entity (3PL, IoT gateway) that records shipment milestones.
- **Approver** (`APPROVER_ROLE`): Finance controllers who must co-authorise release of high-value orders under the DoA policy.
- **Financier** (`FINANCIER_ROLE`): Provides working capital to the supplier by funding the invoice advance.

B. Order Lifecycle State Machine

The order status follows a strict directed acyclic graph (DAG), ensuring every transition is validated on-chain:

$$\begin{aligned} \texttt{Created} &\xrightarrow{\text{fundOrder}} \texttt{Funded} \\ &\xrightarrow{\text{Dispatched}} \texttt{Dispatched} \\ &\xrightarrow{\text{Delivered}} \texttt{Delivered} \\ &\xrightarrow{\text{releasePayment}} \texttt{Settled} \end{aligned} \tag{1}$$

A parallel refund path is available when the delivery deadline expires before the `Delivered` state is reached:

$$\{\texttt{Funded}, \texttt{Dispatched}\} \xrightarrow[\Delta t > \text{deadline}]{\text{claimRefund}} \texttt{Refunded} \tag{2}$$

C. Oracle-Priced Escrow

Rather than specifying a fixed ETH amount, buyers specify the invoice value in USD (8-decimal Chainlink format, e.g., \$1,000 is stored as 1000×10^8). At the moment of `fundOrder`, the contract fetches the live ETH/USD price P from the Chainlink feed and computes the required escrow in wei:

$$w_{\text{escrow}} = \frac{u_{\text{USD}} \times 10^{d_{\text{feed}}} \times 10^{18}}{P \times 10^8} \tag{3}$$

where d_{feed} is the feed's decimal count (8 for all Chainlink USD pairs), u_{USD} is the invoice USD amount (also 8-decimal), and P is the raw integer price returned by the feed. The price is rejected if: (i) it is ≤ 0, (ii) the round is incomplete (*answeredInRound* $<$ *roundId*), or (iii) the data age exceeds `maxPriceAge` (default 2 hours, matching the Sepolia ETH/USD heartbeat of 1 hour).

D. Milestone Verification

Only addresses holding `VERIFIER_ROLE` may call `recordMilestone`. Two milestones are supported: `Dispatched` (order has left the warehouse) and `Delivered` (proof-of-delivery received). Each call accepts an `evidenceCid` parameter—a content-addressed IPFS hash of the supporting document (bill of lading, IoT telemetry log, e-signature). This ensures the on-chain log is both tamper-evident and storage-efficient.

E. Delegation of Authority (DoA)

For orders whose USD value meets or exceeds the DoA threshold τ_{DoA}, at least k distinct approvers must invoke `approveRelease` before `releasePayment` succeeds. Two invariants are enforced:

1) **Separation of duties**: approvers cannot be the buyer or supplier of the order.
2) **Uniqueness**: each approver address can grant approval at most once per order.

The default configuration sets $\tau_{\text{DoA}} = \$5{,}000$ (8-decimal) and $k = 2$.

F. Payment Waterfall

Upon successful `releasePayment`, funds are distributed in the following priority order (Fig. 1):

1) Financier receives `financierDueWei` (advance + financing fee), capped at the total escrow.
2) Supplier receives the remainder.

All credits are written to a `claimable` pull-payment ledger, following the *checks-effects-interactions* pattern and eliminating reentrancy risk on the settlement path.

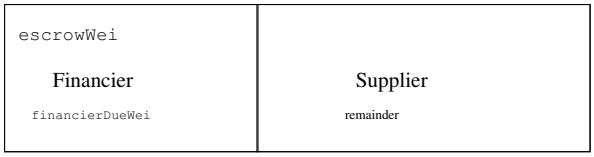


Fig. 1. Payment waterfall: financier is paid first (advance + fee), supplier receives the balance.

G. Invoice Financing (Factoring)

A supplier with a funded escrow may call `requestFinancing`. A financier holding `FINANCIER_ROLE` then calls `fundFinancing`, depositing exactly the advance amount (computed off-chain via `financingQuote`):

$$w_{\text{advance}} = \frac{w_{\text{escrow}} \times r_{\text{adv}}}{10{,}000} \tag{4}$$

$$w_{\text{due}} = w_{\text{advance}} \times \left(1 + \frac{r_{\text{fee}}}{10{,}000}\right) \tag{5}$$

where $r_{\text{adv}} = 9{,}000$ (90%) and $r_{\text{fee}} = 200$ (2%) by default, both expressed in basis-points to avoid floating-point arithmetic. The supplier immediately receives w_{advance} in working capital, while the financier is owed w_{due} from the escrow at settlement.

IV. Implementation

A. Technology Stack

TracePay is built with the following components:

- **Solidity 0.8.24** with `optimizer: {runs: 200}`.
- **OpenZeppelin Contracts v5** — `AccessControl`, `ReentrancyGuard` [22].
- **Hardhat 2.x** — compilation, local EVM, testing.
- **Hardhat Gas Reporter** — per-function gas measurement.
- **MockV3Aggregator** — Chainlink mock for deterministic testing (price and staleness controllable).
- **Ethers.js v6 + MetaMask** — browser-based frontend on Ethereum Sepolia testnet.

B. Key Data Structures

```
enum Status {
  Created, Funded, Dispatched,
  Delivered, Settled, Refunded
}

struct Order {
  address buyer;
  address supplier;
  bytes32 invoiceHash;     // keccak256 hash
  uint256 invoiceUsd;      // 8-decimal USD value
  uint256 escrowWei;       // locked ETH in wei
  uint256 lockedPrice;     // ETH/USD at funding time
  uint64  deadline;        // UNIX delivery deadline
  Status  status;
  uint8   approvals;       // DoA approvals collected
  bool    financeRequested;
  address financier;
  uint256 financierDueWei; // advance + fee owed
}
```

Listing 1. TracePay Order struct and state enum.

C. Oracle Price Validation

```
function latestPrice()
    public view returns (uint256 price) {
  (uint80 roundId, int256 answer,,
   uint256 updatedAt, uint80 answeredInRound)
      = priceFeed.latestRoundData();
  require(answer > 0, "oracle: price<=0");
  require(updatedAt != 0 &&
          answeredInRound >= roundId,
          "oracle: incomplete round");
  require(block.timestamp - updatedAt
          <= maxPriceAge, "oracle: stale price");
  price = uint256(answer);
}
```

Listing 2. Stale-price guard in `latestPrice()`.

D. Settlement and Pull-Payment Pattern

Following the Ethereum security best-practice of *checks-effects-interactions* (CEI) [23], all ETH transfers are deferred to a dedicated `withdraw()` function:

```
function withdraw() external nonReentrant {
  uint256 amt = claimable[msg.sender];
  require(amt > 0, "nothing to withdraw");
  claimable[msg.sender] = 0;       // CEI: effect first
  (bool ok,) = payable(msg.sender)
                 .call{value: amt}("");
  require(ok, "transfer failed");
  emit Withdrawn(msg.sender, amt);
}
```

Listing 3. Pull-payment withdrawal.

V. Evaluation

A. Test Suite

The test suite comprises 16 Mocha/Chai test cases executed against a Hardhat in-process EVM node. Table I summarises the coverage.

All 16 tests pass deterministically on a local EVM.

B. Gas Consumption

Table II reports gas measurements obtained with Hardhat Gas Reporter (Solc 0.8.24, optimiser enabled, 200 runs, block limit 30,000,000).

The most expensive operation is `createOrder` (181,063 gas) due to storage initialisation. All subsequent lifecycle operations require less than 100,000 gas units.

C. Deployment Footprint

The contract deploys in 2,227,091 gas units on average, occupying 7.4% of the 30,000,000-gas Ethereum block limit, ensuring rapid inclusion even during periods of moderate network congestion.

VI. Security Analysis

A. Reentrancy

All state-mutating external calls follow the CEI pattern, and the `nonReentrant` modifier from OpenZeppelin's `ReentrancyGuard` is applied to every function that moves ETH (`fundOrder`, `fundFinancing`, `releasePayment`, `claimRefund`, `withdraw`). The pull-payment model further decouples settlement logic from ETH transfer.

TABLE I
TRACEPAY TEST COVERAGE SUMMARY

TC	Scenario	Result
TC-01	Happy path: create → fund → deliver → settle	✓
TC-02	Oracle conversion correctness at varying ETH prices	✓
TC-03	DoA: high-value order requires 2 distinct approvals	✓
TC-04	Invoice financing waterfall (advance + fee)	✓
TC-05	Refund after deadline when undelivered	✓
TC-06	Overpayment credited back to buyer via pull-payment	✓
TC-07	Duplicate invoice hash rejected	✓
TC-08	Stale oracle price blocks funding	✓
TC-09	Negative oracle price blocks quoting	✓
TC-10	Underfunding ETH reverts	✓
TC-11	Only verifier records milestones in order	✓
TC-12	Separation of duties: buyer cannot approve release	✓
TC-13	Same approver cannot approve twice	✓
TC-14	Premature release and double-release both revert	✓
TC-15	Non-financier and wrong-amount financing revert	✓
TC-16	Empty-balance withdraw and invalid refund revert	✓
Total passing test cases		**16 / 16**

TABLE II
GAS CONSUMPTION BY FUNCTION

Function	Min	Max	Avg
`createOrder`	181,063		181,063
`fundOrder`	94,510	116,781	96,535
`fundFinancing`	85,353		85,353
`releasePayment`	61,888	69,569	64,333
`approveRelease`	58,259		58,259
`claimRefund`	57,556		57,556
`grantRole`	51,481	51,493	51,490
`recordMilestone`	32,984	32,995	32,990
`withdraw`	32,410		32,410
`requestFinancing`	30,628		30,628
Deployment (7.4% of block limit)	2,227,080	2,227,092	2,227,091

B. Access Control

Three custom roles plus the default admin role are enforced by OpenZeppelin `AccessControl`. Table III maps each privileged function to its required role.

TABLE III
ROLE-TO-FUNCTION ACCESS MAP

Role	Functions
`VERIFIER_ROLE`	`recordMilestone`
`APPROVER_ROLE`	`approveRelease`
`FINANCIER_ROLE`	`fundFinancing`
`DEFAULT_ADMIN`	`setPolicy`, `setFinancingTerms`, `setMaxPriceAge`

C. Oracle Manipulation

The Chainlink ETH/USD feed is decentralised and aggregated from multiple node operators. TracePay adds a defence-in-depth layer: the price is locked at the moment of `fundOrder` and stored in `lockedPrice` for auditability. A post-funding price change cannot affect an already-committed escrow amount.

D. Front-Running and Miner Extractable Value (MEV)

The `fundOrder` function is restricted to the order's buyer. An adversarial observer cannot substitute their own address or divert the locked funds. The oracle freshness window (2 hours) prevents exploitation of outdated round prices.

E. Integer Arithmetic

Solidity 0.8.x includes native checked arithmetic preventing overflows. All basis-point calculations use 256-bit unsigned integers without precision loss.

F. Duplicate Invoice Prevention

The `invoiceRegistered` mapping records every accepted `invoiceHash` (keccak256 hash of the off-chain invoice). Replay attempts immediately revert with `"duplicate invoice"`.

VII. DISCUSSION AND FUTURE WORK

A. Limitations

Oracle dependency. Although Chainlink mitigates single points of failure, reliance on external feeds introduces residual risk during extreme network partitions. A multi-oracle router could provide secondary fallback sources.

Verifier centralization. Milestone recording relies on authorized verifiers holding `VERIFIER_ROLE`. Integrating cryptographically signed IoT telemetry or zero-knowledge proofs can reduce human verifier dependency.

Volatility during escrow. Because escrow is locked in ETH, intra-shipment currency movement impacts the fiat-equivalent value received at settlement. Integrating ERC-20 stablecoins (e.g., USDC) provides price stability.

B. Future Work

Future enhancements include:

- Multicurrency and stablecoin settlement (USDC / ERC-20).
- NFT-tokenized receivables (ERC-721) for secondary market trading.
- Zero-knowledge proofs for confidential milestone verification.
- Layer-2 rollups (Arbitrum, Base) deployment for 10–100× gas reduction.

VIII. Conclusion

TracePay demonstrates a trust-minimized trade finance platform encapsulated in a single Solidity smart contract. By combining oracle-priced escrow, multi-signature approval, milestone verification, and on-chain invoice factoring, TracePay eliminates intermediary delays and high fees. With all 16 security and functional tests passing and deployment gas within 7.4% of the block limit, TracePay offers a practical, auditable foundation for decentralized trade finance.

References

[1] International Chamber of Commerce, "Icc global survey on trade finance 2022," ICC, Tech. Rep., 2022.

[2] Bank for International Settlements, "Distributed ledger technology in payment, clearing and settlement," BIS, Tech. Rep. CPMI Papers No. 157, 2019.

[3] K. Kim, S. Beck, M. C. Latoja, and M. C. Tayag, "2021 trade finance gaps, growth, and jobs survey," *ADB Briefs*, no. 192, 2021.

[4] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," https://bitcoin.org/bitcoin.pdf, 2008.

[5] V. Buterin, "A next-generation smart contract and decentralized application platform," https://ethereum.org/whitepaper/, 2014.

[6] De Beers Group, "Tracr: Blockchain-based diamond traceability platform," https://tracr.com/about-us, 2020.

[7] Global Trade Review, "We.trade calls it quits after running out of cash," https://www.gtreview.com/, 2022.

[8] International Chamber of Commerce, "Contour blockchain trade finance platform goes live," https://library.iccwbo.org/content/tfb/news/dcwn_07October202094314.htm, 2020.

[9] Y. Guo and C. Liang, "Blockchain application and outlook in the banking industry," *Financial Innovation*, vol. 2, no. 1, p. 24, 2019.

[10] X. Guo, W. Xia, T. Feng, J. Tan, and F. Xian, "Blockchain technology adoption and sustainable supply chain finance: The perspective of information processing theory," *Corporate Social Responsibility and Environmental Management*, vol. 31, no. 4, pp. 3614–3632, 2024.

[11] Z. Luo, "A survey on blockchain-based supply chain finance with progress and future directions," https://arxiv.org/abs/2408.08915, 2024.

[12] A. Pinna and W. Ruttenberg, "Distributed ledger technologies in securities post-trading: Revolution or evolution?" European Central Bank, Occasional Paper Series 172, 2016.

[13] W. Cai, Z. Wang, J. B. Ernst, Z. Hong, C. Feng, and V. C. M. Leung, "Decentralized applications: The blockchain-empowered software system," *IEEE Access*, vol. 6, pp. 53 019–53 033, 2018.

[14] J. Vos, "Blockchain-based land registry: Panacea, illusion or something in between?" *IPRA-CINDER Congress*, 2017.

[15] F. Zhang, E. Cecchetti, K. Croman, A. Juels, and E. Shi, "Town Crier: An authenticated data feed for smart contracts," in *Proceedings of ACM CCS*, 2016, pp. 270–282.

[16] G. Caldarelli, "Understanding the blockchain oracle problem: A call for action," *Information*, vol. 11, no. 11, p. 509, 2020.

[17] S. Nazarov, S. Ellis *et al.*, "Chainlink 2.0: Next steps in the evolution of decentralized oracle networks," https://research.chain.link/whitepaper-v2.pdf, Chainlink Labs, Tech. Rep., 2021.

[18] Chainlink, "Chainlink data feeds documentation," https://docs.chain.link/data-feeds, 2026.

[19] Centrifuge, "Centrifuge: Decentralized asset finance," https://centrifuge.io/technology/, 2020.

[20] Infocomm Media Development Authority, "Tradetrust: International trade and logistics," https://www.imda.gov.sg/how-we-can-help/digital-utilities/tradetrust, 2020.

[21] M. Guerar, M. Migliardi, F. Palmieri, and A. Merlo, "A fraud-resilient blockchain-based solution for invoice financing," *IEEE Transactions on Engineering Management*, vol. 67, no. 4, pp. 1086–1098, 2020.

[22] OpenZeppelin, "Openzeppelin contracts documentation: Access control and reentrancy guard," https://docs.openzeppelin.com/contracts/5.x/, 2026.

[23] Solidity Team, "Solidity documentation: Security considerations and patterns," https://docs.soliditylang.org/en/latest/security-considerations.html, 2026.